

Basic Matrix Algebra

This notebook presents some basic linear algebra and some matrix reshuffling/computations often needed in finance and econometrics.

```
using Printf, LinearAlgebra  
  
include("src/printmat.jl");
```

Adding and Multiplying: An Array and a Scalar

With an array A (for instance, a vector or a matrix, but also higher-dimensional arrays) and a scalar c , do

1. $A * c$ (textbook: Ac) to multiply each element of A by c
2. $A .+ c$ (textbook: $A + cJ$, where J is an array of ones) to add c to each element of A , and similarly $A .- c$ (textbook: $A - cJ$). Notice the dot.

Watch out when the number comes first: $2.*A$ is not allowed since it is ambiguous. However, $2.*A$ and $2 .* A$ both work.

```
A = [1 3; 3 4]  
c = 10  
  
printblue("A:")  
printmat(A)  
printblue("c:")  
printmat(c)  
  
printblue("A*c:")  
printmat(A*c)
```

```
printblue("A .+ c:")
printmat(A .+ c)      #notice the dot in .+
```

```
A:
    1      3
    3      4
```

```
c:
    10
```

```
A*c:
    10      30
    30      40
```

```
A .+ c:
    11      13
    13      14
```

Adding and Multiplying Two Arrays

With two arrays of the same dimensions (A and B), do

$A+B$ (textbook: $A + B$) to add them (element by element), and similarly $A-B$ (textbook: $A - B$).

To multiply arrays (A and B) of conformable dimensions: $A*B$ (textbook: AB).

```
A = [1 3;3 4]      #A and B are 2x2 matrices
B = [1 2;3 -2]
printblue("A:")
printmat(A)
printblue("B:")
printmat(B)

printblue("A+B:")
printmat(A+B)

printblue("A*B:")
printmat(A*B)
```

```
A:
    1      3
```

	3	4
B:		
	1	2
	3	-2
A+B:		
	2	5
	6	2
A*B:		
	10	-4
	15	-2

Transpose

You can transpose a numerical vector/matrix A by A' .

Notice that (in Julia) A and $B = A'$ share the same elements (changing one changes the other). If you want an independent copy, use $B = \text{permutedims}(A)$ or perhaps $B = \text{copy}(A')$.

For an array of other elements (for instance, strings), use $\text{permutedims}(A)$ to swap the dimensions.

```
A = [1 2 3; 4 5 6]
printblue("A: ")
printmat(A)
printblue("A': ")
printmat(A')
```

A:			
	1	2	3
	4	5	6
A':			
	1	4	
	2	5	
	3	6	

Vectors: Inner and Outer Products

There are several different ways to think about a vector in applied mathematics: as a $K \times 1$ matrix (a column vector), a $1 \times K$ matrix (a row vector) or just a flat K vector. Julia uses flat vectors but they are mostly interchangeable with column vectors.

The inner product of two (column) vectors with K elements is calculated as $x'z$ or `dot(x,y)` (textbook: $x'z$ or $x \cdot z$) to get a scalar. (The dot is obtained by `\cdot` + TAB, but this is sometimes hard to distinguish from or things like $x \cdot z$.)

In contrast, the outer product of two (column) vectors with K elements is calculated as xz' (textbook: xz') to get a $K \times K$ matrix.

```
x = [10,11]          #[10;11] gives the same
z = [2,5]
printblue("x and z:")
printmat(x,z)

printblue("x'z: ")
printlnPs(x'z)        #dot(x,z) gives the same

printblue("x*z':")
printmat(x*z')
```

x and z:

10	2
11	5

x'z:

75

x*z':

20	50
22	55

Vectors: Quadratic Forms

A quadratic form (A is an $n \times n$ matrix and x is an n vector): $x'A*x$ (textbook: $x'Ax$) to get a scalar. There is also the function `dot(x,A,x)`.

```

A = [1 3;3 4]
x = [10,11]

printblue("x:")
printmat(x)
printblue("A:")
printmat(A)

printblue("x'A*x: ")
printlnPs(x'A*x)      #or dot(x,A,x)

```

x:

```

10
11

```

A:

```

1      3
3      4

```

x'A*x:

```

1244

```

Kronecker Product

The Kronecker product of two matrices

$$A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{bmatrix}$$

is calculated by using `kron()`

```

A = [1 3;2 4]
B = [10 11]
z = kron(A,B)

printblue("A:")
printmat(A)
printblue("B:")
printmat(B)

```

```
printblue("kron(A,B):")
printmat(z)
```

A:

1	3
2	4

B:

10	11
----	----

kron(A,B):

10	11	30	33
20	22	40	44

The Identity Matrix

The identity matrix I_n can often be represented by `I` and then Julia will compare with the surrounding code to create the right dimension. For instance, if `A` is a square matrix, then `I + A` and `[A I]` both work. Notice that you need to do using `LinearAlgebra` before `I` works.

If you still need to specify the dimension, then use `1I(3)` or `1I[1:3,1:3]`.

You can create a basis vector as `1I[1:3,2]`. This 3-vector is filled with zeros, except that element 2 is 1.

```
printblue("I*A and A*I")
printmat(I*A,A*I)
```

```
printblue("I + A")
printmat(I + A)
```

```
printblue("1I[1:3,1:3]")
printmat(1I[1:3,1:3])
```

```
printblue("1I[1:3,2]")
printmat(1I[1:3,2])
```

I*A and A*I

1	3	1	3
2	4	2	4

```
I + A
      2      3
      2      5
```

```
1I[1:3,1:3]
      1      0      0
      0      1      0
      0      0      1
```

```
1I[1:3,2]
      0
      1
      0
```

Matrix Inverse

The rank of a matrix is the number of linearly independent columns.

A matrix inverse of an $n \times n$ matrix A :

`inv(A)` or A^{-1} (textbook: A^{-1})

The inverse is such that $AA^{-1} = I$ and $A^{-1}A = I$, where I is the identity matrix (ones along the main diagonal, zeros elsewhere).

```
A = [1 3; 3 4]
printblue("A:")
printmat(A)

printblue("rank(A):")
printlnPs(rank(A), "\n")

printblue("inv(A):")
printmat(inv(A))

printblue("inv(A)*A:")
printmat(inv(A)*A)
```

```
A:
      1      3
      3      4
```

```
rank(A):
```

2

```
inv(A):  
  -0.800    0.600  
  0.600   -0.200
```

```
inv(A)*A:  
  1.000   -0.000  
  0.000    1.000
```

Solving Systems of Linear Equations

If you have the system $Ab = y$, where A is a matrix and b and y are vectors, then we can solve as $b = \text{inv}(A)*y$ or $b = A \backslash y$. The latter is typically more robust.

```
y = [10,11]  
  
b1 = inv(A)*y  
b2 = A \ y  
  
printmat(b1,b2,colNames=["sol 1","sol 2"])  
  
printblue("checking the solution: is A*b1-y=0?")  
printmat(A*b1-y)
```

```
sol 1    sol 2  
-1.400   -1.400  
 3.800    3.800
```

```
checking the solution: is A*b1-y=0?  
 0.000  
 0.000
```

Eigenvalues and Eigenvectors

using the `svd()` function, since it automatically sorts in descending order.


```

Ω = [5 2;
     2 2]

F = svd(Ω)
λ = F.S      #eigenvalues
W = F.U      #vectors

printmat(λ,W;colNames=["λ","W1","W2"])

tst = W*diagm(λ)W'      #does it recreate Ω?
printmat(tst-Ω)

```

λ	W ₁	W ₂
6.000	-0.894	-0.447
1.000	-0.447	0.894
0.000	0.000	
0.000	0.000	

Vectors: Extracting Vectors from Matrices

Notice that $A[1,:]$ and $A[:,1]$ both give flat vectors. In case you want a row vector use $A[1:1,:]$.

```
A[1,:]
```

2-element Vector{Int64}:

```
1
3
```

Optimization

of linear-quadratic problems, using the [OSQP.jl](#) package. An alternative, [Clarabel.jl](#) is discussed briefly at the end of the notebook.

The methods illustrated here are well suited for cases when the objective is linear-quadratic, for instance, the portfolio variance ($w'\Sigma w$) or when the estimation problem is based on minimizing the sum of squared residuals ($u'u$), and the restrictions are linear expressions.

Load Packages and Utility Functions

```
using Printf, LinearAlgebra, SparseArrays, OSQP

include("src/printmat.jl");
```

```
using Plots, LaTeXStrings      #LaTeXStrings to facilitate using LaTeX in plots
default(size = (480,320),fmt = :png)
```

The OSQP.jl Optimization Package

The [OSQP.jl](#) package is tailor made for solving linear-quadratic problems (with linear restrictions). It solves problems of the type

$$\min 0.5\theta'P\theta + q'\theta \text{ subject to } l \leq A\theta \leq u,$$

where θ is a vector of choice variables.

To get an equality restriction in row i , set $l[i]=u[i]$.

Notice that (P,A) should be Sparse matrices and (q,l,u) vectors with Float64 numbers.

We consider several cases below: no restrictions on θ , bounds on θ , and a linear equality restriction.

Unconstrained Minimization

We will define the following minimization problem

$$P = 2 \begin{bmatrix} 1 & 0 \\ 0 & 16 \end{bmatrix}$$

and

$$q = \begin{bmatrix} -4 \\ 24 \end{bmatrix}.$$

It is straightforward to see that this is the same as minimizing $(\theta_1 - 2)^2 + (4\theta_2 + 3)^2$

If there are no constraints, then the solution should be $\theta = (2, -3/4)$.

Before doing the optimization, we plot the contours of the loss function.

```
P = 2*[1 0;      #we minimize 0.5*theta'P*theta + q'*theta, hence the 2*[]
        0 16]    #this is the same as minimizing (x-2)^2 + (4y+3)^2
q = [-4.0, 24.0] #vector, Float64

function Lossfn(P,q,theta)
    L = 0.5*theta'*P*theta + q'*theta
    return L
end

(n1,n2) = (82,122)
theta1 = range(1,5,length=n1)
theta2 = range(-1,0,length=n2)

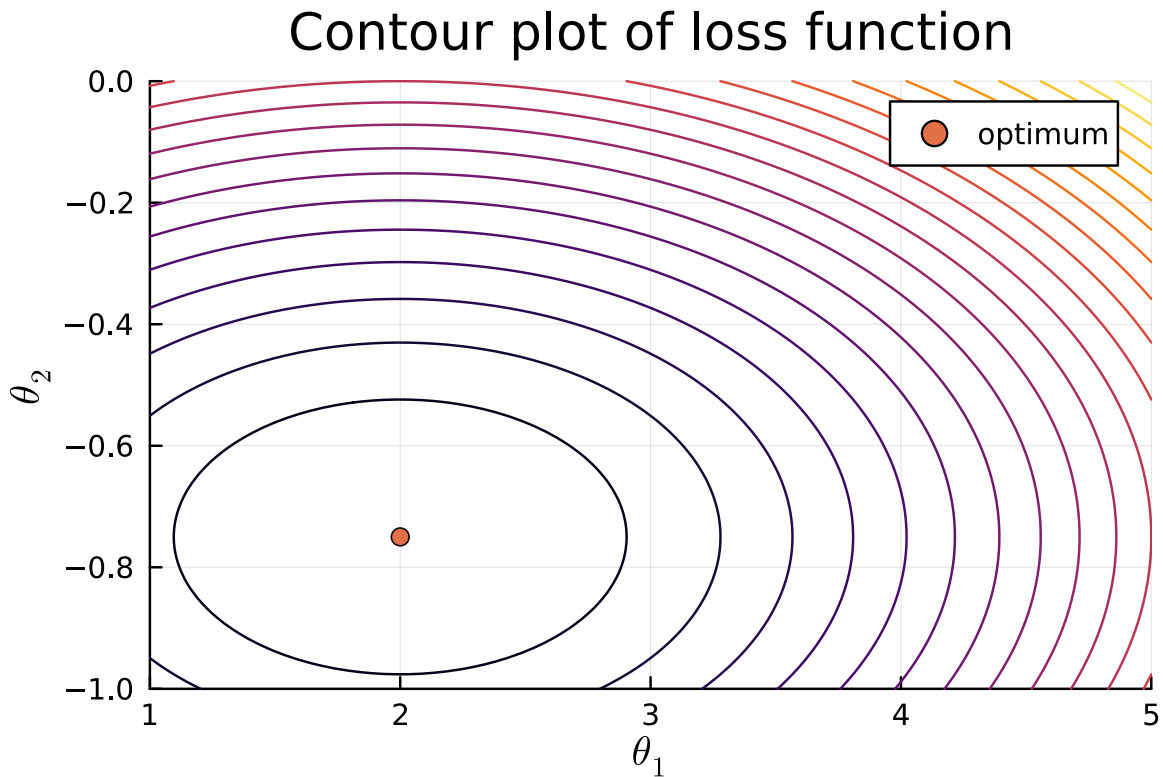
loss2d = fill{NaN}(n1,n2) #matrix with loss fn values
for i in 1:n1, j in 1:n2
    loss2d[i,j] = Lossfn(P,q,[theta1[i],theta2[j]])
end

p1 = contour(theta1,theta2,loss2d', #notice: loss2d'
             xlims = (1,5),
             ylims = (-1,0),
             legend = false,
             levels = 21,
             title = "Contour plot of loss function",
             xlabel = L"\theta_1",
```

```

ylabel = L"\theta_2" )
scatter!([2],[-0.75],label="optimum",legend=true)
display(p1)

```



```

A = zeros(1,2)          #effectively no restriction
l = [0.0]               #vectors, Float64
u = [0.0]

settings = Dict{:verbose => true}          #to print more info on iterations
model = OSQP.Model()                      #create model
OSQP.setup!(model; P=sparse(P), q=q, A=sparse(A), l=l, u=u, settings...)  #set up model, P and A s
result = OSQP.solve!(model)               #solve

printblue("Unconstrained minimization: the solution should be (2,-3/4):")
printmat(result.x)
printblue("\nand lots of information about the iterations:")

```

Unconstrained minimization: the solution should be (2,-3/4):

2.000
-0.750

and lots of information about the iterations:

```
-----  
OSQP v0.6.2 - Operator Splitting QP Solver  
  (c) Bartolomeo Stellato, Goran Banjac  
University of Oxford - Stanford University 2021  
-----  
problem: variables n = 2, constraints m = 1  
         nnz(P) + nnz(A) = 2  
settings: linear system solver = qdldl,  
         eps_abs = 1.0e-003, eps_rel = 1.0e-003,  
         eps_prim_inf = 1.0e-004, eps_dual_inf = 1.0e-004,  
         rho = 1.00e-001 (adaptive),  
         sigma = 1.00e-006, alpha = 1.60, max_iter = 4000  
         check_termination: on (interval 25),  
         scaling: on, scaled_termination: off  
         warm start: on, polish: off, time_limit: off  
  
iter  objective    pri res    dua res    rho        time  
   1  -8.3200e+000  0.00e+000  1.44e+001  1.00e-001  7.10e-005s  
  25 -1.3000e+001  0.00e+000  6.82e-005  1.00e-001  7.75e-005s  
  
status:                solved  
number of iterations: 25  
optimal objective:     -13.0000  
run time:               8.03e-005s  
optimal rho estimate: 1.00e-006
```

Constrained Minimization

Bounds on the solution: $2.75 \leq \theta_1$ and $\theta_2 \leq -0.3$.

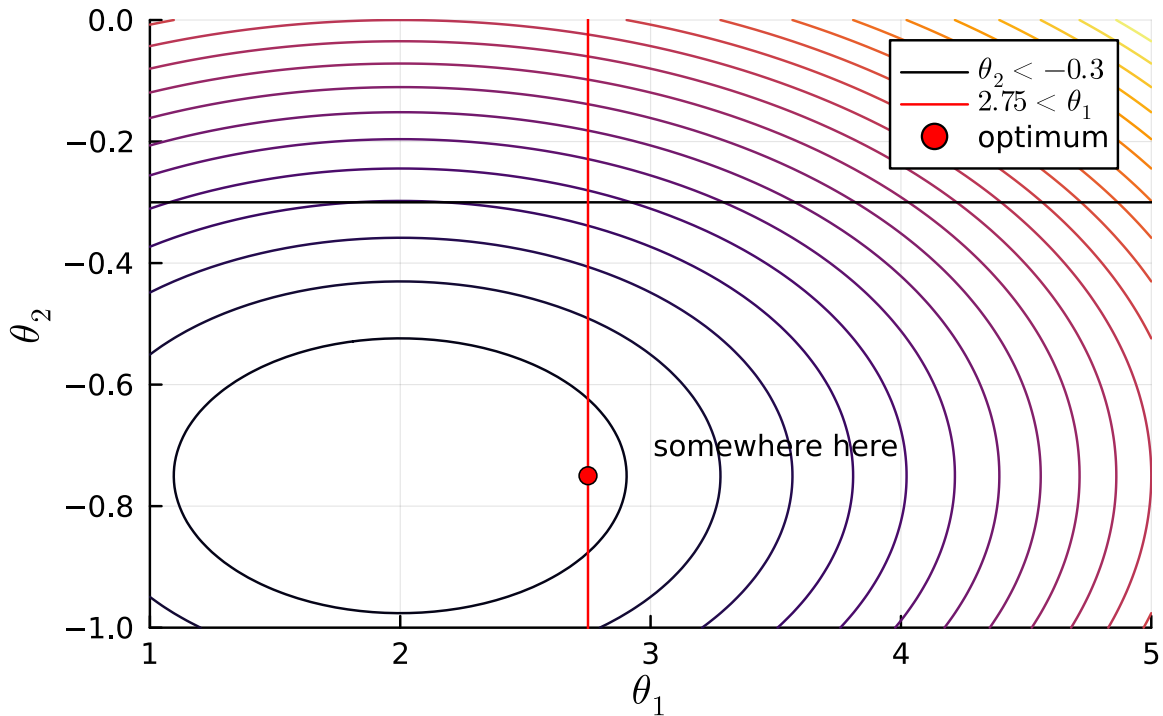
```
p1 = contour(  $\theta_1, \theta_2$ , loss2d',  
             xlims = (1,5),  
             ylims = (-1,0),  
             legend = false,  
             levels = 21,  
             title = "Contour plot of loss function",  
             xlabel = L"\theta_1",
```

```

        ylabel = L"\theta_2",
        annotation = (3.5,-0.7,text("somewhere here",8)) )
hline!([-0.3],linecolor=:black,label=L"\theta_2 < -0.3")
vline!([2.75],linecolor=:red,line=:solid,label=L"2.75 < \theta_1")
scatter!([2.75],[-0.75],color=:red,label="optimum",legend = true)
display(p1)

```

Contour plot of loss function



```

A = I[1:2,1:2]      #identity matrix, I2
l = [2.75,-Inf]      #lower bounds on A*x
u = [Inf,-0.3]       #upper bounds, 2.75 <= θ1 <= Inf, -Inf <= θ2 <= -0.3

settings = Dict(:verbose => false)
model = OSQP.Model()
OSQP.setup!(model; P=sparse(P), q=q, A=sparse(A), l=l, u=u, settings...)
result = OSQP.solve!(model)

printblue("with bounds on the solution: the solution should be (2.75,-0.75):")
printmat(result.x)

```

```
printblue("checking if the bounds are satisfied:")
printmat(l,A*result.x,u;colNames=["l","Aθ","u"])
```

with bounds on the solution: the solution should be (2.75,-0.75):

```
2.750
-0.750
```

checking if the bounds are satisfied:

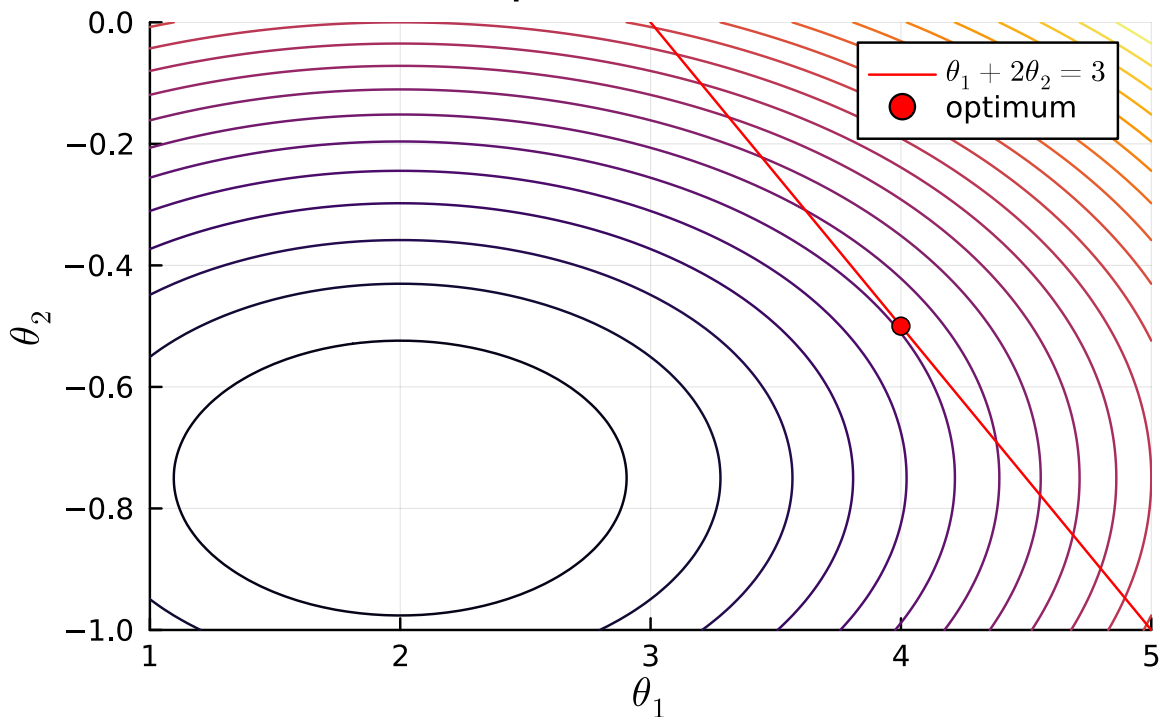
l	Aθ	u
2.750	2.750	Inf
-Inf	-0.750	-0.300

Constrained Minimization

A linear equality constraint: $\theta_1 + 2\theta_2 = 3$.

```
p1 = contour(θ1,θ2,loss2d',
             xlims = (1,5),
             ylims = (-1,0),
             legend = false,
             levels = 21,
             title = "Contour plot of loss function",
             xlabel = L"\theta_1",
             ylabel = L"\theta_2" )
Plots.abline!(-0.5,1.5,linecolor=:red,line=:solid,label=L"\theta_1+2\theta_2=3")
scatter!([4],[-0.5],markercolor=:red,label="optimum",legend=true)
display(p1)
```

Contour plot of loss function



```
A = [1 2]                                #equality constraint
l = [3.0]
u = [3.0]

model = OSQP.Model()
OSQP.setup!(model; P=sparse(P), q=q, A=sparse(A), l=l, u=u, settings...)
result = OSQP.solve!(model)

printblue("equality constraint: the solution should be (4,-1/2):\n")
printmat(result.x)
printblue("checking if the bounds are satisfied:")
printmat(l,A*result.x,u;colNames=["l","Aθ","u"])
```

equality constraint: the solution should be (4,-1/2):

```
4.000
-0.500
```

checking if the bounds are satisfied:

l	$A\theta$	u
3.000	3.000	3.000

Updating the Problem (extra)

When you just want to change some of the inputs to the optimization problem, then it may pay off to do `update!()`. This is especially useful when you resolve the problem many times (in a loop, say).

```
model = OSQP.Model()           #setting up the problem for the first time
OSQP.setup!(model; P=sparse(P), q=q, A=sparse(A), l=l, u=u, settings...)
result = OSQP.solve!(model)
printmat(result.x)

q_new = [-2.0, 20.0]           #resolve the problem with different q values
OSQP.update!(model; q=q_new)   #(partially) update the problem
result = OSQP.solve!(model)
printmat(result.x)
```

```
4.000
-0.500
```

```
3.600
-0.300
```

An Alternative Package (extra)

The [Clarabel.jl](#) package is an interesting alternative to OSQP.jl. It has a slightly different syntax, but the `DoClarabel()` function in `src/DoClarabel.jl` provides a way to call it using the same syntax as for OSQP.jl.

```
using LinearAlgebra, SparseArrays, Clarabel
include("src/DoClarabel.jl");
```

```
(x,result,fn0) = DoClarabel(P,q,A,l,u,Clarabel.Settings(verbose = false))
printblue("equality constraint: the solution should be (4,-1/2), as before:")
printmat(x)
```

equality constraint: the solution should be $(4, -1/2)$, as before:

4.000

-0.500

Numerical Optimization

This notebook uses the [Optim.jl](#) package for unconstrained and box constrained (lower and upper bounds).

For other types of constraints, consider [Optimization.jl](#), [NLOpt.jl](#), or [JuMP.jl](#).

The optimization problems are (for pedagogical reasons) the same as in the other notebook about optimization. This means that the solutions should be very similar and that the contour plots in the other notebook can be used as references.

However, the current notebook is focused on methods for solving general optimization problems. In contrast, the other notebook focuses on linear-quadratic problems (mean-variance, least squares, etc), where there are faster algorithms.

Load Packages and Extra Functions

```
using Printf, Optim

include("src/printmat.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Optimization with One Choice Variable

Running

```
Sol = optimize(x->fn1(x,0.5),a,b)
```

finds the x value (in the interval $[a,b]$) that *minimizes* $fn1(x,0.5)$. The $x \rightarrow fn1(x,0.5)$ syntax makes this a function of x only, which is what the `optimize()` function wants.

The `optimize()` is here called without specifying any options. This means that defaults will be used, for instance, for the algorithm choice.

The output (Sol) contains a lot of information

If you prefer the SciML convention of allowing for parameters in the loss function, then consider the wrapper of `Optim.jl` at [Optimization.jl](#).

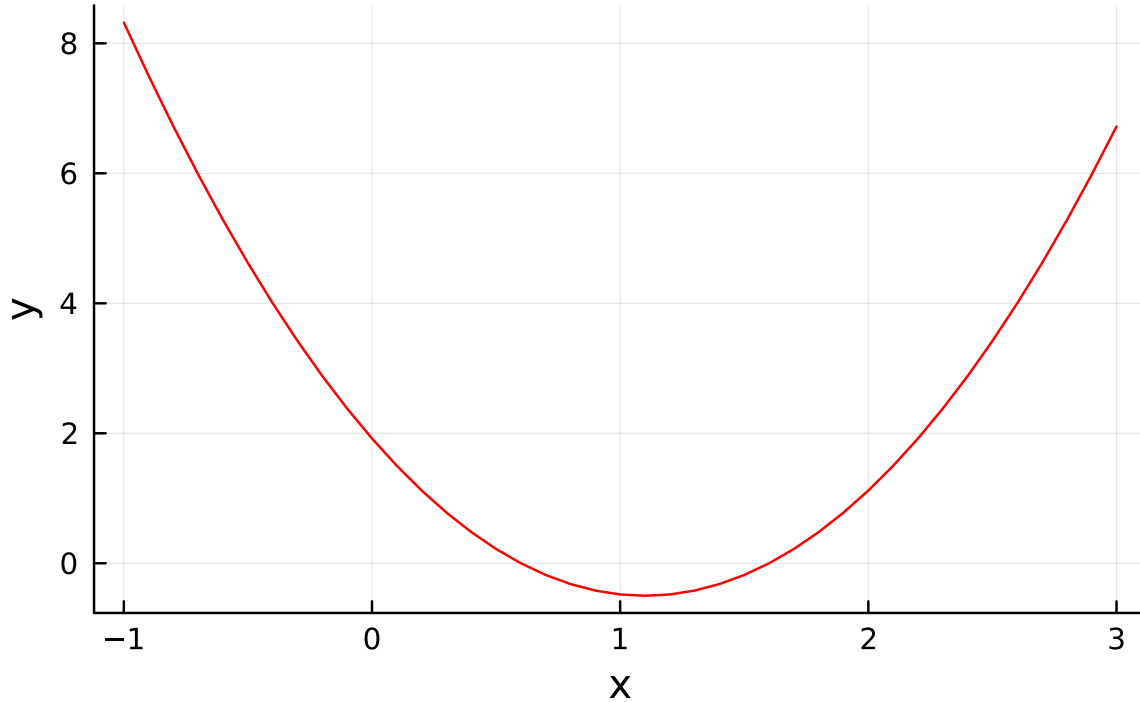
```
function fn1(x,c)                                #notice: the function has two arguments
    value = 2*(x - 1.1)^2 - c
    return value
end
```

fn1 (generic function with 1 method)

```
x = -1:0.1:3

p1 = plot( x,fn1.(x,0.5),                        #plotting the function can help identify
           linecolor = :red,                     #starting points and possible issues
           linewidth = 1,
           legend = nothing,
           title = "the fn1(x,0.5) function",
           xlabel = "x",
           ylabel = "y" )
display(p1)
```

the fn1(x,0.5) function



```
Sol = optimize(x→fn1(x,0.5),-2.0,3.0) #using a default algorithm
println(Sol)

printblue("\nThe minimum is at: ", Optim.minimizer(Sol)) #the optimal x value
printblue("Compare with the plot above")
```

Results of Optimization Algorithm

```
* Status: success
* Algorithm: Brent's Method
* Search Interval: [-2.000000, 3.000000]
* Minimizer: 1.100000e+00
* Minimum: -5.000000e-01
* Iterations: 5
* Convergence: max(|x - x_upper|, |x - x_lower|) <= 2*(1.5e-08*|x|+2.2e-16): true
* Objective Function Calls: 6
```

The minimum is at: 1.1

Compare with the plot above

One Choice Variable: Supplying a Starting Guess Instead (extra)

If you prefer to give a starting guess x_0 instead of an interval, then supply it as a vector $[x_0]$. We also use `x→fn1(only(x),0.05)` to make sure that the `fn1()` function gets a scalar, not a 1-element vector. The solution is then also a vector (with one element.)

```
x0 = 1.0
Solb = optimize(x→fn1(only(x),0.5),[x0])

printlnPs("The minimum is at: ",Optim.minimizer(Solb))
```

The minimum is at: 1.100

Several Choice Variables: Unconstrained Optimization

In the example below, we choose (x,y) so as to minimize the fairly simple objective function

$$(x + c_1)^2 + (4y + c_2)^2,$$

without any constraints. We will use $(c_1, c_2) = (-2, 3)$ in the example below, so the solution should be $(x,y) = (2, -3/4)$.

The `BFGS()` algorithm is good (and quick) for smooth problems. Otherwise, use the default (Nelder-Mead()). Unless we specify a function (or algorithm) for the gradient, `optimize()` will calculate it numerically. This typically works well for problems with a small/moderate number of choice variables.

```
function fn2(θ,c)
    (x,y) = (θ[1],θ[2])          #unpack the choice variables and get nicer names
    L      = (x+c[1])^2 + (4*y+c[2])^2
    return L
end
```

fn2 (generic function with 1 method)

```
c = [-2,3]
θ0 = [4.0,-0.5]                #starting guess

Sol = optimize(u→fn2(u,c),θ0,BFGS()) #BFGS() is one of several algorithm choices

printlnPs("minimum at (x,y)= ",Optim.minimizer(Sol))
```

minimum at (x,y)= 2.000 -0.750

Several Choice Variables: Bounds on the Solutions

The next few cells discuss how to impose bounds on the solution.

In the example below, we impose $2.75 \leq x$ (a lower bound) and $y \leq -0.3$ (an upper bound). We will see that only one of these restrictions binds, as the solution should be $(x, y) = (2.75, -3/4)$.

We here use `Fminbox(BFGS())` to use `BFGS()` but let `optimize()` make sure that the bounds are respected.

```
lower_θ = [2.75, -Inf]
upper_θ = [Inf, -0.3]

Sol = optimize(u→fn2(u,c),lower_θ,upper_θ,θ₀,Fminbox(BFGS()))
printlnPs("The optimum is at (x,y) = ",Optim.minimizer(Sol))
```

The optimum is at (x,y) = 2.750 -0.750

Several Choice Variables: Supplying the Gradient (extra)

Supplying a function for calculating the derivatives improves speed and accuracy. See below for an example.

```
function g2!(G,θ,c)                                #derivatives of fn2 wrt. θ[1] and θ[2]
    (x,y) = (θ[1],θ[2])
    G .= (2*(x+c[1]), 2*4*(4*y+c[2]))              #the tuple () fills/updates an existing G vector, create
    return G
end

Sol3 = optimize(u→fn2(u,c),(G,u)→g2!(G,u,c),[1.0,-0.5],BFGS())
println(Sol3)
printblue("Probably faster than the solution above")
printblue("Minimum at: ",round.(Optim.minimizer(Sol3),digits=3))
```

* Status: success

* Candidate solution

Final objective value: 0.000000e+00

* Found with

Algorithm: BFGS

* Convergence measures

$ x - x' $	$= 9.34e-01 \not\leq 0.0e+00$
$ x - x' / x' $	$= 4.67e-01 \not\leq 0.0e+00$
$ f(x) - f(x') $	$= 8.75e-01 \not\leq 0.0e+00$
$ f(x) - f(x') / f(x') $	$= \text{Inf} \not\leq 0.0e+00$
$ g(x) $	$= 0.00e+00 \leq 1.0e-08$

* Work counters

Seconds run:	0 (vs limit Inf)
Iterations:	2
f(x) calls:	5
$\nabla f(x)$ calls:	5
$\nabla f(x)^T v$ calls:	0

Probably faster than the solution above

Minimum at: [2.0, -0.75]